# TP 5 Web Service REST

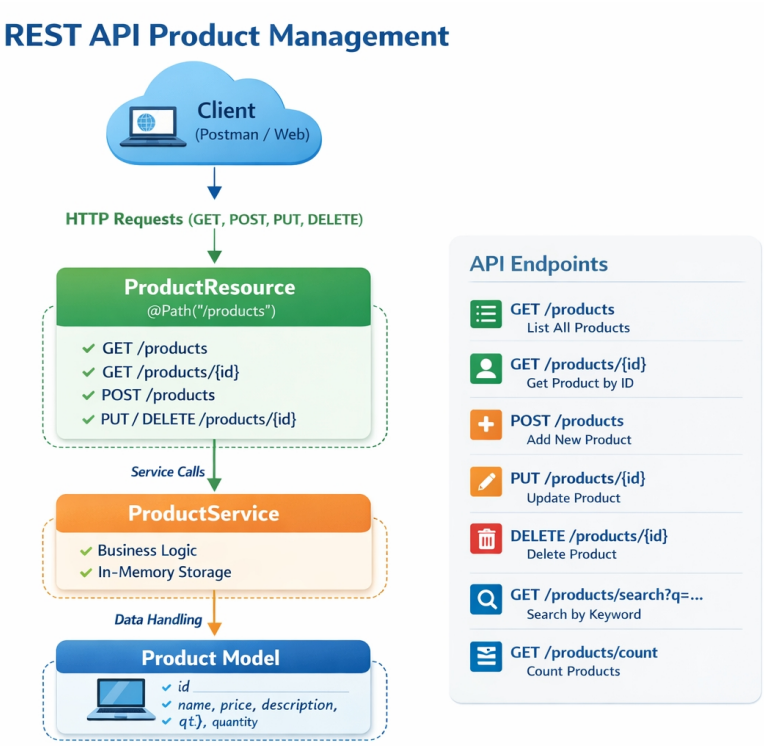
**Ce TP à pour objectif de :**

- Comprendre l'architecture d'un Web Service REST (Model/ Service/ Resource)
- Implémenter un Web Service REST avec **JAX-RS**
- Publier le web service REST
- Tester le service avec **SoapUI/Postman**
- Générer et consommer un client REST

## I. Étude de cas : REST API Product management :

Dans ce TP, on vous fournit le code source d'une API REST développée avec JAX-RS permettant de gérer un catalogue de produits (consultation, ajout, modification, suppression, recherche, etc.).

Cependant, certaines parties du code ont été volontairement supprimées ou incomplètes. Votre objectif est de comprendre l'architecture de l'application (modèle, service, ressource) puis de compléter les fonctionnalités manquantes en respectant les principes REST et les bonnes pratiques (gestion des méthodes HTTP, codes de réponse, validation des données).




1. Télécharger le projet fourni (rest-service)
2. Décompresser dans votre répertoire de travail
3. Importer le projet (projet maven existant) dans votre IDE

   Le projet doit contenir :
   - `src/main/java` → code source
   - `src/main/resources` → configuration éventuelle
   - `pom.xml` → gestion Maven

3. Tenter de compiler / exécuter l'application pour vérifier qu'il n'y a pas d'erreurs de dépendances.

4. Démarrer le serveur et vérifier que l'api est accessible via le lien : http://localhost:9090/api/products

5. Identifier le rôle de chaque classe de l'application.

6. Quelle annotation permet de définir une ressource REST ? Où est-elle utilisée dans le projet ?

7. Quelles sont les méthodes HTTP utilisées dans cette API ? Donner un exemple pour chacune.

8. Compléter la méthode `deleteProduct` dans `ProductService` pour supprimer un produit en fournissant son ID et retourne true après ( false sinon)

9. Compléter la méthode getProductById dans `ProductResource` qui permet de retourner un produit selon son id.

10. A quoi sert la classe Response et comment l'utiliser ?

11. Utiliser Postman ou navigateur pour tester le endpoint `/products` :
    - GET → liste tous les produits
    - POST → ajouter un nouveau produit (vérifier que le nom n'est pas vide)
    - etc.

12. Accéder au fichier WADL généré pour l'API REST et Complétez le tableau suivant en indiquant le nom trouvé dans le fichier et son rôle dans l'API REST.

```
http://localhost:9090/api/application.wadl
```

| Élément WADL | Nom trouvé / valeur | Rôle / Description |
|---|---|---|
| <application> | | Racine du document WADL, contient toutes les ressources et méthodes |
| <resources> | | Conteneur des ressources (ex : /products) |
| <resource> | | Décrit un endpoint spécifique et son chemin relatif |
| <method> | | Méthode HTTP utilisée (GET, POST, PUT, DELETE) |
| <request> | | Paramètres attendus pour la requête (queryParam, pathParam, formParam) |
| <response> | | Type de réponse attendue (representation) |
| <representation> | | Format des données (ex : application/json) |
| <doc> | | Documentation associée à la ressource ou méthode |
| <param> | | Description et type des paramètres (nom, type, style : query/path/form) |

## II Étude de cas : magasin ElectroShop

Une entreprise souhaite créer un service web REST pour gérer ses ventes. Les données seront ensuite utilisées pour des analyses BI (tableau de bord, total par produit/région) et AI (prévision de ventes). Exemple d'une vente

```
{
  "id": 1,
  "date": "2026-03-20",
  "product": "Laptop HP",
  "quantity": 2,
  "price": 5500.0,
  "total": 11000.0,
  "customer": "Ali Benjelloun",
  "region": "Rabat"
}
```

1. Créez une classe Sale avec tous les champs listés ci-dessus.
2. Créez une classe SaleService pour gérer les ventes en mémoire (ex : ConcurrentHashMap<Long, Sale>). Implémentez les méthodes suivantes :
   - List<Sale> getAllSales()
   - Sale getSaleById(Long id)
   - Sale addSale(Sale sale) → générer l'ID si absent, calculer total
   - Sale updateSale(Long id, Sale sale)
   - boolean deleteSale(Long id)
   - int countSales()
3. Créez une classe SaleResource annotée avec @Path("/sales"). Implémentez les endpoints suivants :
   - GET /sales → lister toutes les ventes
   - GET /sales/{id} → récupérer une vente par ID
   - POST /sales → ajouter une vente
   - PUT /sales/{id} → mettre à jour une vente
   - DELETE /sales/{id} → supprimer une vente
   - GET /sales/count → retourner le nombre total de ventes

Lancez le serveur intégré et testez chaque endpoint avec Postman ou navigateur en utilisant plusieurs ventes d'exemple.

4. Analyser le fichier WADL
   - Accédez à http://localhost:9090/api/application.wadl et Identifiez les chemins /sales et /sales/{id}.
   - Vérifiez les méthodes HTTP et les représentations JSON.
   - Reliez le WADL aux méthodes que vous avez implémentées.

5. Swagger  est un outil qui permet de documenter automatiquement les API REST.Préparer une documentation Swagger pour votre API REST de ventes.
   - Ajoutez Swagger à votre projet (ex : Maven ou dépendance Gradle).
   - Annoter votre classe SaleResource pour :
     - Décrire l'API globale (@Api)
     - Décrire chaque méthode (@ApiOperation)
     - Documenter les paramètres (@ApiParam) et les réponses (@ApiResponse)
   - Lancer le serveur et ouvrir Swagger UI et Vérifiez que :
     - Tous les endpoints (/sales, /sales/{id}, /sales/count) apparaissent correctement
     - Les descriptions des paramètres et types de retour sont claires